차례

PART 1 전체 구조와 개발 환경

1장	웹 애플리케이션의 구조	버전 1
2장	개발 환경: 터미널, 가상환경, .env, 서버 실행	버전 1

PART 2 데이터베이스

3장	관계형 데이터베이스의 기본 개념	버전 1
4장	이음 스키마 한 줄씩 읽기	버전 1
5장	SQL: 조회, 입력, 수정, 삭제, JOIN, 집계, 트랜잭션	버전 1

PART 3 서버: Python, FastAPI, pandas

6장	main.py에 나온 Python 문법	버전 1
7장	FastAPI로 API 만들기	버전 1
8장	이음의 핵심 로직 3가지	버전 1
9장	pandas로 연구 데이터 다루기	버전 2

PART 4 규칙 · AI · 예측

10장	경고 규칙	버전 2
11장	LLM 안내 Agent	버전 2
12장	머신러닝: 추적검사 이탈 위험 예측	버전 2
13장	딥러닝: 공여자 문장 분류	버전 2

PART 5 화면과 도구

14장	HTML, CSS, JavaScript	버전 1 · 2
15장	Git과 GitHub로 이력 관리·공개하기	버전 1 · 2

PART 6 정리

16장	실제로 만난 오류와 해결	버전 1 · 2
17장	꼭 알아야 할 것 체크리스트	버전 1 · 2
18장	용어 사전	버전 1 · 2
19장	확인 문제와 정답	버전 1 · 2

PART 7 실제 서비스로 가는 길

20장	공여자 화면 개선: 하루 기록, 측정 기준, 기준값 설명, 검사 여정	버전 3
21장	로그인과 보안	버전 3
22장	EMR 연동: FHIR · LOINC	버전 3
23장	eCRF로 옮기기	버전 3
24장	휴대폰 앱: React Native, 매일 알림, 블루투스	버전 3
25장	실제로 쓸 때: 법 · 비용, 확인 문제 32~39	버전 3

버전 1 · 2 는 버전 1에 있던 장에 버전 2 내용을 더한 장입니다. 버전 3에서 앞 장이 바뀐 점은 20.6에 정리했습니다.

이 책을 읽는 법

이 책은 생체 신장 공여자 추적관리 앱 **이음**을 만들면서 사용한 기술을 정리한 학습서입니다. 모든 예시는 이음의 실제 코드와 데이터(공여자 D-001, D-002, D-003)입니다.

개념 정의와 원리 **꼭 알아야 할 것** 면접·실무에서 묻는 내용 **주의** 실수하면 데이터나 보안에 문제가 생김 **확인 방법** 직접 실행해 보는 법

코드는 **회색 바탕 상자**에, 코드를 한 줄씩 풀어 쓴 부분은 **왼쪽 코드 · 오른쪽 해설** 표로 정리했습니다. 처음 읽을 때는 각 장의 **꼭 알아야 할 것** 상자만 먼저 읽어도 전체 흐름을 잡을 수 있습니다.

PART 7

실제 서비스로 가는 길

버전 3에서 더한 내용입니다. 공여자가 이해하기 쉬운 화면, 여러 사람이 쓰기 위한 **로그인**, 병원 시스템과 잇는 **EMR 연동(FHIR)**과 **eCRF**, 어르신도 쓰기 쉬운 **휴대폰 앱**, 그리고 실제로 쓸 때 확인해야 할 **법·비용**을 정리합니다. 앞 장의 일부 내용(주 1회 기록, 로그인 없음)은 이 PART에서 바뀌었습니다(20.6).

20장	공여자 화면 개선: 하루 1회 기록, 측정 기준, 저혈압 판정, 기준값 API, 검사 여정
21장	로그인과 보안: 비밀번호 해시, 세션, 권한(401 · 403)
22장	EMR 연동: FHIR · LOINC, 모의 EMR, 중복 없는 자동 저장
23장	eCRF로 옮기기: REDCap 가져오기 형식, 변수 정의서, 코드화
24장	휴대폰 앱: React Native, 매일 소리 알림, 블루투스 혈압계
25장	실제로 쓸 때: 법 · 알림 비용, 확인 문제

20장 공여자 화면 개선

공여자는 의료인이 아닙니다. "eGFR 88"만 보여 주면 좋은지 나쁜지 알 수 없습니다. 버전 3은 **무슨 기록인지**, **기준이 무엇인지**, **내 수치가 어디쯤인지**를 화면에서 바로 알 수 있게 바꿨습니다.

20.1 바뀐 화면 한눈에

요청	바꾼 내용
어떤 기록인지 모름	카드마다 한 줄 설명. 혈압은 "앞 숫자(수축기) · 뒤 숫자(이완기)", 소변 단백 Negative → **음성**
측정 기준 · 경고	주 1회 → **하루 1회** 기록. 기록 창에 언제·어떻게 재는지. 저장 직후 정상 · 높음 · 매우 높음 · **낮음** 판정
검사 수치 설명	검사 칸을 누르면 기준표에서 **내 수치 줄이 강조**되고 기증자용 설명이 나옴
추적검사 설명 부족	"검사 여정 자세히": 기증일 → 6개월 → 1년 → 2년 → 3년 → 매년, 단계별 의미 · 검사 기간
크레아티닌 그래프	크레아티닌 / eGFR 변화 그래프 + 기준 점선
채팅이 어려움	"채팅 시작하기" 버튼, 오른쪽 아래 떠 있는 "이음 안내" 버튼, 채팅 창 위 추천 질문
신장 그림	기증한 신장 · "남은 신장 1개" 표시를 지우고 "나의 신장" 하나만 표시

20.2 언제, 어떻게 재나요: 측정 기준

항목	기준
시간	아침에 일어나서 1시간 안, 소변을 본 뒤, 약 먹기 전 · 아침 식사 전. 저녁(잠들기 전)은 선택
준비	재기 30분 전에는 커피 · 담배 · 운동을 피한다. 등을 기대고 앉아 5분 쉬고, 팔은 심장 높이
횟수	1~2분 간격으로 2번 재서 평균. 매일 같은 시간에 재야 비교할 수 있다
체중	아침에 소변을 본 뒤, 식사 전, 가벼운 옷차림

개념 · 가정혈압 기준이 병원 기준과 다른 이유

병원에서는 긴장 때문에 혈압이 높게 나오는 경우가 많아(백의 고혈압), 집에서 잰 혈압은 더 낮은 기준을 씁니다. 이음은 가정혈압 고혈압 기준 **135/85**를 씁니다. 저혈압은 **90/60 미만**을 기준으로 "어지러우면 연락"을 안내합니다(데모 기준값, 의료진 승인 필요).

20.3 bp_assess: 저장 직후 판정

코드	해설
`def bp_assess(sys_, dia) -> dict:`	혈압 한 번의 값을 판정하는 **순수 함수**. DB를 보지 않는다(10.2와 같은 이유).
`    if sys_ >= BP_CRISIS_SYS or dia >= BP_CRISIS_DIA:` `        return {"level": "crisis", ...}`	180/110 이상 → 5분 쉬고 다시, 증상이 있으면 119. **가장 위험한 조건부터** 검사한다.
`    if sys_ >= BP_HOME_SYS or dia >= BP_HOME_DIA:` `        return {"level": "high", ...}`	135/85 이상 → 다시 재 보고, 며칠 계속되면 알리기.
`    if sys_ < BP_LOW_SYS or dia < BP_LOW_DIA:` `        return {"level": "low", ...}`	90/60 미만 → 물 마시고 앉아서 다시. 어지러우면 연락.
`    return {"level": "normal", ...}`	정상 범위.
`return {"home_id": ..., "updated": ...,` `        "bp": bp_assess(m.systolic_bp, m.diastolic_bp)}`	POST `/home-measurements` 응답에 판정을 함께 돌려준다 → 화면은 저장 직후 결과 카드를 보여 준다.

저혈압 규칙은 경고 목록(`donor_alerts`)에도 더했습니다. 가장 최근 혈압이 낮으면 `bp-low-날짜` 경고(warn)가 생기고, 공여자 알림함에는 "혈압이 낮게 측정되었어요"가 뜹니다. 기록 알림도 하루 단위로 바뀌어 id가 `daily-2026-11-04`처럼 날짜를 품습니다. 같은 날에는 같은 id라서 "새 알림"이 하루 한 번만 생깁니다.

20.4 기준값 API: /guide/reference

eGFR · 크레아티닌 · 소변 단백 · 혈압의 쉬운 이름, 설명, 기준표, 기증자용 설명을 서버의 `REFERENCE` 한 곳에 두고 API로 내보냅니다. 웹 화면과 휴대폰 앱이 같은 내용을 쓰고, 안내 Agent의 안내문(크레아티닌 · 소변 단백 주제 추가)과도 맞춥니다.

검사	쉬운 이름	기준 (일반 성인 참고)
eGFR	콩팥 기능 점수	90 이상 정상 · 60~89 약간 감소 · 45~59 · 30~44 · 30 미만 진료 필요. **기증 후에는 기증 전의 60~70%가 흔함**
크레아티닌	콩팥이 걸러 내는 노폐물	남성 약 0.7~1.3, 여성 약 0.5~1.1 mg/dL. eGFR과 반대로 움직임
소변 단백	소변에 새어 나온 단백질	음성 정상 · 미량 다시 확인 · 1+ 양성(정밀검사 ACR) · 2+ 이상 빨리 확인
혈압	앞 숫자 · 뒤 숫자	135/85 미만 정상 · 이상 높음 · 180/110 이상 매우 높음 · 90/60 미만 낮음

화면에서는 같은 기준으로 칩(정상 · 기증 후 흔함 · 확인 필요 · 양성)을 붙이고, 설명 창에서 `lv.row`에 해당하는 줄에 `class="me"`를 붙여 강조합니다. 크레아티닌 기준은 성별에 따라 다르므로 `/summary` 응답에 `sex`를 더했습니다.

20.5 검사 여정 · 그래프 · 채팅 바로가기

코드 (donor.html)	해설

`function refLine(rows, y, color, label) {`
`  const xs = rows.map((r) => r.measured_at ?? r.tested_at);`
`  return { label, data: [{x: xs[0], y}, {x: xs.at(-1), y}],`
`  borderDash: [5, 4], pointRadius: 0 };`
`}`
	기준선(점선)을 **점 두 개짜리 선**으로 그린다. 첫 날짜와 마지막 날짜를 같은 높이로 이으면 수평선이 된다. 추가 플러그인 없이 Chart.js 기본 기능만 쓴다.

`legend: { labels: { filter: (i) => !i.text.startsWith("기준") } }`	범례에서 기준선은 숨긴다.
`const VISIT_TEXT = { "기증 후 6개월": "남은 신장이 ... 적응하는 시기", ... }`	단계별 설명. 일정·상태는 서버의 `visits`를 쓰고, 검사 기간은 예정일 ± `grace_days`(60일).

`function startChat() {`
`  openChat();`
`  if (!$("chatLog").children.length) addMsg("bot", "안녕하세요 ...");`
`}`
	질문을 고르지 않고 바로 채팅 창을 연다. "채팅 시작하기" 버튼과 떠 있는 버튼(`position: fixed`)이 같은 함수를 부른다.

20.6 앞 장에서 바뀐 점

앞 장	버전 3에서
8.3 알림 계산	"이번 주 기록" 알림 → "오늘 기록" 알림 (`daily-날짜`)
10.1 경고 규칙	저혈압(90/60 미만) 규칙 추가
11장 안내 Agent	안내문 주제 크레아티닌 · 소변 단백 추가, "이번 주" 문구 → "오늘"
14장 화면	로그인 화면(`login.html`)과 로그아웃 추가, 데모 공여자 선택은 의료진만
7.5 · 9장 · 검사 입력	입력자는 직접 적지 않고 로그인한 의료진 이름으로 저장. 엑셀 시트 5개 → 7개(eCRF 2개)

꼭 알아야 할 것 · 설계 이유

- **기준값은 서버 한 곳**(`REFERENCE`, 상수): 웹 · 앱 · Agent가 같은 기준을 쓴다.
- **판정은 알려 주되 진단하지 않는다**: "범위 안 / 다시 확인 / 연락"까지만. 판단은 의료진.
- **기증자 맞춤 설명**: 일반 기준만 보여 주면 eGFR 60대가 "감소"로만 보여 불안을 키운다.

21장 로그인과 보안

여러 공여자가 같은 서버를 쓰면 **내 기록은 나만** 봐야 합니다. 버전 3은 회원가입 · 로그인 · 세션을 만들고, 모든 API에 "누가 무엇을 볼 수 있는가"를 붙였습니다(`auth.py`).

21.1 가입 규칙과 테이블

가입	확인하는 것
공여자	병원이 알려 준 **공여자 코드 + 출생 연도**가 donors 테이블과 같아야 함. 한 공여자에 계정 하나 (`donor_id UNIQUE`)
의료진	`.env` 의 **STAFF_INVITE_CODE**(병원 가입 코드)를 알아야 함. 비어 있으면 의료진 가입이 닫힘

```
CREATE TABLE users (
    user_id SERIAL PRIMARY KEY, username TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL CHECK (role IN ('donor', 'staff')),
    display_name TEXT NOT NULL, donor_id INTEGER UNIQUE REFERENCES donors(donor_id),
    CONSTRAINT donor_has_donor_id CHECK (role <> 'donor' OR donor_id IS NOT NULL));
CREATE TABLE sessions (token_hash TEXT PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    expires_at TIMESTAMPTZ NOT NULL);
```

두 테이블은 서버가 시작할 때 `CREATE TABLE IF NOT EXISTS` 로 자동으로 만듭니다(`migrate_auth.sql` 과 같은 내용). `CHECK` 제약으로 "공여자 계정은 반드시 donor_id가 있다"를 데이터베이스가 지키게 했습니다.

21.2 비밀번호는 해시로만 저장

코드 (auth.py)	해설

`salt = secrets.token_hex(16)`
`digest = hashlib.pbkdf2_hmac("sha256", password.encode(),`
`salt.encode(), 200_000).hex()`
`return f"pbkdf2_sha256$200000${salt}${digest}"`
	소금(salt): 사람마다 다른 임의 문자열. 같은 비밀번호도 저장값이 달라진다. **20만 번 반복**: 한 번 계산이 느려서 무작위 대입이 어렵다. 원문 비밀번호는 어디에도 저장하지 않는다.

`test = hashlib.pbkdf2_hmac(...)  # 같은 소금·횟수로 다시 계산` `return hmac.compare_digest(test, digest)`	로그인 때 입력값으로 다시 계산해 비교. `compare_digest` 는 비교 시간이 항상 같아서 "어디까지 맞았는지"를 시간으로 추측할 수 없다.
`if v.isdigit() or v.isalpha():` `raise ValueError("글자와 숫자를 섞어 8자 이상")`	Pydantic 검사(7.3): 숫자만 · 글자만인 비밀번호는 422.

개념 · 해시

입력을 정해진 길이의 값으로 바꾸는 계산입니다. 같은 입력은 항상 같은 결과가 나오지만, 결과에서 입력을 되돌릴 수 없습니다. 그래서 DB가 유출되어도 비밀번호 원문은 드러나지 않습니다.

21.3 세션과 쿠키

단계	하는 일
로그인 성공	`secrets.token_urlsafe(32)` 로 임의 토큰을 만들고, DB에는 토큰의 **SHA-256 해시**와 만료일(14일)만 저장
웹	토큰을 **HttpOnly** 쿠키로 준다. 자바스크립트가 읽을 수 없어 페이지에 나쁜 코드가 끼어도 훔치기 어렵다. `SameSite=Lax`, HTTPS면 `Secure`
휴대폰 앱	요청 머리글 `X-Client: mobile` 이면 응답에 토큰을 담아 주고, 앱은 `Authorization: Bearer 토큰` 으로 보낸다(24장)
요청마다	토큰 해시로 sessions를 찾아 만료 전인지 확인 → 사용자 · 역할 · 공여자 코드
로그아웃	sessions에서 그 행을 지우고 쿠키 삭제

같은 아이디로 10분 안에 5번 틀리면 429(요청이 너무 많음)를 돌려 잠시 막습니다. 공여자 가입에서 "코드가 없음"과 "출생 연도가 틀림"을 같은 문장으로 답하는 것도, 남의 공여자 코드를 하나씩 맞춰 보지 못하게 하려는 것입니다.

21.4 권한: Depends로 API마다 붙이기

코드 (main.py)	해설

`def current_user(request: Request) -> dict:`
`  user = auth.user_from_request(conn, reques t)`
`  if user is None: raise HTTPException(401, ...)`
	로그인 안 됨 → **401**.

`def staff_only(user = Depends(current_user)):` `  if user["role"] != "staff": raise HTTPExcep tion(403, ...)`	로그인은 했지만 권한이 없음 → **403**.

`def donor_access(donor_code: str, user = Depend s(current_user)):`
`  if user["role"] != "staff" and user["donor_ code"] != donor_code:`
`  raise HTTPException(403, "본인 기록만 볼 수 있습니다")`
	경로의 `{donor_code}` 를 FastAPI가 같은 이름의 매개변수로 넘겨 준다. 의료진은 모두, 공여자는 자기 코드만.

`@app.get("/donors/{donor_code}/summary",` `  dependencies=[Depends(donor_access)])`	함수 본문을 고치지 않고 경로에 검사만 붙인다. 의료진 전용 API(`/staff/overview` , 엑셀, EMR)는 `Depends(staff_only)` .

확인: 공여자 계정으로 자기 요약 200, 다른 공여자 요약 403, 의료진 API 403, 다른 공여자 기록 저장 403, 로그아웃 뒤 401, 다시 로그인 200.

꼭 알아야 할 것 · 면접 질문 대비

- **401 vs 403**: 401 = 누구인지 모름(로그인 필요), 403 = 누구인지는 알지만 허락 안 됨.
- **비밀번호 · 토큰 원문은 저장하지 않는다**: 둘 다 해시만. 비밀번호는 느린 해시(PBKDF2) + 소금.
- **권한은 서버에서**: 화면에서 버튼을 숨기는 것은 보안이 아니다. API마다 검사한다.
- **남은 일**: 누가 어떤 기록을 봤는지 남기는 감사 로그, HTTPS 배포.

22장 EMR 연동: FHIR

지금은 연구 담당자가 검사 결과를 손으로 옮겨 적습니다. 병원 EMR(전자의무기록)에서 바로 가져오면 옮겨 적는 실수와 시간이 줄어듭니다. 실제 EMR은 병원 승인 없이 연결할 수 없어서, **국제 표준 FHIR 형식의 모의 EMR**을 만들고 같은 코드가 실제 주소로도 동작하게 했습니다(`emr.py`).

22.1 FHIR와 LOINC

개념 · FHIR

HL7이 만든 의료정보 교환 표준입니다. 환자(`Patient`), 검사 결과(`Observation`) 같은 자료를 **리소스**라는 정해진 JSON 모양으로 주고받고, 웹 API로 검색합니다. 여러 결과는 `Bundle` 에 담겨 옵니다.

LOINC 코드	검사	이음 항목
2160-0	Creatinine [Mass/volume] in Serum or Plasma	creatinine
98979-8 · 62238-1	eGFR (CKD-EPI 2021 · 이전 식)	egfr
20454-5	Protein [Presence] in Urine by Test strip	urine_protein

LOINC는 검사마다 붙인 국제 코드입니다. 병원마다 검사 이름이 달라도("Cr", "크레아티닌") 코드는 같아서, 이름 대신 코드로 어떤 검사인지 구분합니다.

```
{"resourceType": "Observation", "status": "final",
 "code": {"coding": [{"system": "http://loinc.org", "code": "2160-0"}]},
 "subject": {"identifier": {"system": "urn:ieum:donor-code", "value": "D-002"}},
 "effectiveDateTime": "2026-10-20T09:00:00+09:00",
 "valueQuantity": {"value": 107.9, "unit": "umol/L"}}
```

22.2 parse_bundle: FHIR → 이음 검사 결과

코드 (emr.py)	해설
`if r.get("status") not in ("final", "amended", "corrected"): continue`	확정된 결과만. 예비 결과(preliminary)는 버린다.
`kind = next((LOINC[c] for c in codes if c in LOINC), None)`	LOINC 코드로 어떤 검사인지. 모르는 검사는 건너뜀.
`row = by_day.setdefault(when, {...})`	Observation은 검사 하나당 하나 → **검사일별로 묶어** 이음의 한 행(크레아티닌 · eGFR · 소변 단백)으로 만든다.
`if unit in ("umol/L", "µmol/L"):` `  value = round(value / 88.4, 2)`	크레아티닌 단위 변환: µmol/L ÷ 88.4 = mg/dL. 107.9 → 1.22. 모르는 단위는 저장하지 않고 `problems` 에 적는다.
`if not 0 < value < 20: problems.append(...)`	화면 입력과 같은 범위 검사(LabIn). 말이 안 되는 값은 저장 전에 거른다.

22.3 save_lab: 손 입력과 자동 가져오기가 같은 함수

검사 결과 저장 · 추적검사 자동 완료(8.2)를 `save_lab()` 하나로 묶고, 의료진 입력(`POST /labs`)과 EMR 가져오기(`sync_emr`)가 모두 이 함수를 씁니다. 저장 전에 같은 날짜의 결과를 먼저 읽어 **new**(새로 저장) · **updated**(값이 바뀜) · **same**(같은 값 → 건너뜀)으로 나눕니다. 그래서 여러 번 가져와도 결과가 중복되지 않습니다(멱등성). 입력자는 "EMR 자동(FHIR)"으로 남습니다.

공여자	모의 EMR 자료	가져온 결과 (실제 확인)
D-001	2025-06-22(이미 있음), 2026-07-08 1년 검사	새 1 · 이미 있음 1 · **1년 검사 완료 처리**
D-002	2026-10-20 크레아티닌 107.9 µmol/L	1.22 mg/dL로 변환 · **6개월 검사 완료**
D-003	새 결과 없음	이미 있음 1
두 번째	같은 자료를 한 번 더 가져옴	세 명 모두 새 0 · 바뀜 0 (중복 없음)

22.4 모의 EMR에서 실제 EMR로

설정 (.env)	동작
EMR_FHIR_BASE 비움	emr.py 안의 모의 자료를 바로 읽음
EMR_FHIR_BASE=http://127.0.0.1:8000/mock-emr/fhir	HTTP로 FHIR 검색: `GET /Observation?patient.identifier=…|D-001&category=laboratory`
EMR_FHIR_TOKEN	실제 EMR이 발급한 접근 토큰을 `Authorization: Bearer` 로
EMR_SYNC_MINUTES=60	60분마다 자동으로 가져옴(0이면 의료진 화면 버튼으로만)

주의 · 실제 병원 EMR 연결에 필요한 것

병원 정보보호 · 의료정보팀 승인, 보안 심사, 인증 방식(SMART on FHIR Backend Services 등), 병원 내부망 또는 전용 연결, 접근 기록. 포트폴리오에서는 **표준 형식을 이해하고 같은 코드가 주소만 바꾸면 동작하게 설계**한 것을 보여 줍니다.

23장 eCRF로 옮기기

임상연구 자료는 결국 eCRF(전자 증례기록서)에 들어갑니다. 엑셀을 보고 칸마다 다시 입력하면 시간이 들고 실수가 생깁니다. 버전 3은 **eCRF가 바로 읽을 수 있는 형식**으로 내보냅니다(`ecrf.py`). 기준은 연구 현장에서 널리 쓰는 REDCap의 가져오기(Import) 형식입니다.

23.1 입력용 표: 대상자 × 방문 한 줄

열	내용
record_id	공여자 번호(가명). 이름·연락처는 넣지 않는다
redcap_event_name	방문: baseline_arm_1(기본 정보), month6_arm_1, year1_arm_1, year2_arm_1, year3_arm_1
birth_year · sex · donation_date · pre_egfr	baseline 줄에만
visit_due · visit_date · visit_status	방문 줄: 예정일, 시행일(미시행이면 빈칸), 상태 코드
scr · egfr · uprot	그 방문과 연결된 검사 결과
home_sbp · home_dbp · home_n · weight	시행일 전 14일 가정혈압 평균 · 평균에 쓴 횟수 · 마지막 체중

```
record_id,redcap_event_name,birth_year,sex,donation_date,pre_egfr,visit_due,visit_date,visit_st
atus,scr,egfr,uprot,...
D-001,baseline_arm_1,1980,2,2025-06-15,135,,,,,,,,...
D-001,month6_arm_1,,,,,2025-12-15,2025-12-20,1,1.02,88,0,...
```

23.2 코드화와 변수 정의서

변수	코드
sex	1 남 · 2 여
uprot	0 음성 · 1 미량 · 2 1+ · 3 2+ · 4 3+
visit_status	1 완료 · 2 검사 기간 · 3 예정 · 4 놓침
날짜	YYYY-MM-DD 글자 (date_ymd)

변수 정의서(`DICTIONARY`)는 REDCap Data Dictionary 열 이름(Variable / Field Name, Form Name, Field Type, Field Label, Choices, Text Validation …)을 그대로 써서, eCRF를 새로 만들 때 그대로 올릴 수 있습니다. 허용 범위(Min · Max)를 함께 적어 eCRF가 입력값을 검사하게 합니다.

코드 (ecrf.py)	해설
`h = home[(home["donor_code"] == v.donor_code) & (home["measured_at"] <= done) & (home["measured_at"] > done - timedelta(days=14))]`	pandas 조건 필터(9.2): 그 공여자의, 시행일 전 14일 기록만.
`out[col] = pd.to_numeric(out[col]).astype("Int64")`	빈칸을 허용하는 정수형. 그냥 int로 바꾸면 빈칸이 있을 때 오류, float이면 "1.0"으로 써진다.
`df.to_csv(index=False).encode("utf-8-sig")`	BOM이 붙은 UTF-8: 엑셀에서 열어도 한글이 깨지지 않는다.

내려받기 (의료진 화면)	용도
연구 데이터 엑셀 (시트 7개)	기존 5개 + eCRF 입력용 + eCRF 변수정의
/export/ecrf.csv?kind=data	eCRF 가져오기(Import)에 바로 올리는 CSV
/export/ecrf.csv?kind=dictionary	변수 정의서 CSV

꼭 알아야 할 것 · CRC 경험과 연결

- **eCRF는 숫자 코드와 정해진 변수 이름**을 쓴다. 화면용 한글 표와 연구용 표를 따로 만든 이유.
- **가명 번호만**: 직접 식별 정보를 빼야 연구용으로 넘기기 쉽다(25.1).
- **정의서가 먼저**: 변수 이름 · 코드 · 허용 범위를 정해 두면 자료 검증(data query)이 줄어든다.

24장 휴대폰 앱

웹 화면은 주소를 찾아 들어가야 하고, 화면이 닫혀 있으면 알림도 울리지 않습니다. 버전 3은 **React Native(Expo)**로 공여자 앱을 만들었습니다. 같은 FastAPI 서버를 쓰고, 큰 글씨 · 큰 버튼 · 매일 소리 알림 · 블루투스 기기 연결을 더했습니다(`mobile_src/`).

24.1 React Native와 Expo

개념 · React Native

자바스크립트(React)로 안드로이드 · 아이폰 앱을 함께 만드는 도구입니다. HTML의 `div` · `p` 대신 `View` · `Text` · `TextInput` · `Pressable`을 쓰고, CSS 대신 `StyleSheet.create({...})`로 모양을 정합니다. **Expo**는 설치 · 실행을 쉽게 해 주는 도구로, 휴대폰의 Expo Go 앱으로 QR 코드를 찍어 바로 실행합니다.

화면	내용
로그인	서버 주소(같은 와이파이의 PC 주소) · 아이디 · 비밀번호. 토큰은 **SecureStore**(휴대폰 보안 저장소)
홈	오늘 기록 여부, 다음 검사, 혈압 판정, 검사 결과(쉬운 이름 + 칩), 크레아티닌 막대그래프
오늘 기록	측정 방법, 큰 숫자 칸, 저장 직후 bp_assess 결과 카드
이음 안내	추천 질문 + 채팅 (`POST /donors/{code}/chat`)
설정	매일 알림 시각(30분 단위), 로그아웃

코드 (App.js)	해설
`const [screen, setScreen] = useState("loading");`	**상태(state)**: 지금 어느 화면인지. 값이 바뀌면 화면이 다시 그려진다. 화면 이동 라이브러리 없이 상태 하나로 전환.
`useEffect(() => { load(); }, [load]);`	화면이 처음 나타날 때 서버에서 자료를 불러온다.
`headers: { Authorization: `Bearer ${token}` }`	앱은 쿠키 대신 토큰을 머리글로 보낸다(21.3). 401이면 `AuthError` → 로그인 화면.

24.2 매일 소리 알림

```
await Notifications.setNotificationChannelAsync("daily", {
  name: "매일 측정 알림", importance: Notifications.AndroidImportance.HIGH,
  sound: "default", vibrationPattern: [0, 400, 200, 400] });
await Notifications.scheduleNotificationAsync({
  content: { title: "이음 · 혈압과 체중을 잴 시간이에요", sound: "default" },
  trigger: { type: DAILY, hour, minute, channelId: "daily" } });
```

휴대폰 안에서 예약하는 **로컬 알림**이라 서버 · 인터넷 · 비용 없이 매일 같은 시각에 울립니다. 안드로이드는 알림 **채널**의 중요도를 HIGH로 해야 소리와 화면 위 알림이 나옵니다. 기록을 빠뜨린 공여자에게 따로 알리는 일(알림톡 · 문자)은 서버가 맡아야 합니다(25.2).

24.3 블루투스 혈압계 · 체중계

기기	표준 BLE 규격 (GATT)
혈압계	Blood Pressure 서비스 **0x1810** / 측정값 0x2A35
체중계	Weight Scale 서비스 **0x181D** / 측정값 0x2A9D

코드 (bleParse.js)	해설

`let mantissa = raw & 0x0fff, exponent = raw >> 12;`
`if (mantissa >= 0x0800) mantissa -= 0x1000;`
`return mantissa * Math.pow(10, exponent);`
	혈압은 **SFLOAT**(16비트) 숫자: 아래 12비트 가수, 위 4비트 지수(둘 다 부호 있음). 0x0078 → 120, 0xF4B5 → $1205 \times 10^{-1} = 120.5$.

`const flags = b[0]; if (flags & 0x01) ... kPa → mmHg`	첫 바이트 flags의 비트로 단위 · 시각 · 맥박 포함 여부를 안다.
`const kg = flags & 0x01 ? raw * 0.01 * 0.4536 : raw * 0.005;`	체중: kg이면 0.005 kg 단위, 파운드면 0.01 lb 단위.

주의 · 블루투스의 한계

표준 규격을 지원하는 기기만 연결됩니다(제조사 전용 규격 기기는 별도 SDK 필요). 블루투스 모듈은 Expo Go에서 쓸 수 없어 **개발 빌드**가 필요합니다. 웹은 안드로이드 크롬만 Web Bluetooth를 지원하고 아이폰 사파리는 지원하지 않습니다. 받은 값은 바로 저장하지 않고 칸에 채워 **본인이 확인한 뒤 저장**합니다.

25장 실제로 쓸 때: 법 · 비용

이 장은 2026년 11월 기준으로 찾아본 일반 정보이며 법률 자문이 아닙니다. 실제 운영 전에는 병원 법무 · 정보보호 담당, IRB와 확인해야 합니다.

25.1 확인해야 할 법과 대응

법 · 제도	이음에서 문제될 수 있는 점	대응
개인정보보호법 (민감정보)	혈압 · 검사 결과는 건강정보 = 민감정보	일반 개인정보와 **별도 동의**. 목적 · 항목 · 보유 기간 고지
개인정보보호법 (가명정보)	연구 활용	가명처리(공여자 번호만) 후 과학적 연구 목적. eCRF 형식이 이미 가명 번호만 씀
생명윤리법	사람 대상 연구 · 자료 수집	**IRB 승인** 후 수집 · 활용
개인정보 국외 이전	안내 Agent가 Groq(해외 서버)에 질문 · 기록을 보냄	국외이전 동의, 또는 식별 정보 제외 · 국내 / 병원 내부 모델(Ollama)
의료법	앱이 진단 · 처방을 하면 원격의료 문제	안내 · 기록 · 의료진 연결까지만. "판단은 의료진" 유지. 비대면진료는 개정 의료법으로 2026.12.24 시행 예정이나 의료인이 하는 진료에 대한 것
의료기기법 · 디지털의료제품법	AI가 응급 여부를 분류해 경고 → 의료기기(SaMD)로 볼 여지	연구용 · 의료진 보조로 한정, 출시 전 식약처 사전 상담. 디지털의료제품법 2차 시행 2026.1.24

25.2 문자 · 카카오 알림톡 비용

방법	건당 (예시)	특징
앱 로컬 알림	0원	휴대폰 안에서 매일 같은 시각. 앱 설치 필요
카카오 알림톡	약 13원	친구 추가 없이 전화번호로. 정보성만. 사업자 · 비즈니스 채널 · 템플릿 심사 필요
문자 SMS · LMS	약 18원 · 45원	카카오톡을 안 쓰는 분. 알림톡 실패 시 대체 발송

대행사 공개 단가(VAT 별도) 예시이며 계약 · 발송량에 따라 다릅니다. 공여자 100명에게 매일 1번 알림톡을 보내면 한 달 약 3,000건 × 13원 ≈ 4만 원입니다. 권장 구성: **앱 알람(매일) + 기록이 없을 때만 알림톡 + 카톡이 없으면 문자**.

25.3 확인 문제 (버전 3)

32 가정혈압 기준이 병원 진료실 기준보다 낮은 이유와, 이음의 가정혈압 · 저혈압 기준값은?

33 기증 후 eGFR이 65인 공여자에게 "감소"라고만 보여 주면 안 되는 이유는?

34 비밀번호를 저장할 때 소금(salt)과 20만 번 반복을 쓰는 이유를 각각 쓰시오.

35 401과 403의 차이를 이음의 예로 설명하시오.

36 FHIR에서 검사 이름 대신 LOINC 코드로 검사를 구분하는 이유는?

37 EMR 가져오기를 여러 번 해도 결과가 중복되지 않는 이유는? (save_lab)

38 eCRF 입력용 표에서 성별 · 소변 단백을 숫자 코드로 바꾸는 이유와, 정수 열에 Int64를 쓴 이유는?

39 안내 Agent가 해외 LLM 서버를 쓸 때 확인해야 할 법적 문제와 대응 두 가지는?

25.4 정답 32 ~ 39

32 병원에서는 긴장으로 혈압이 높게 나오기 쉬워(백의 고혈압) 가정혈압은 낮은 기준을 쓴다. 가정혈압 135/85 이상 높음, 180/110 이상 매우 높음, 90/60 미만 낮음. (20.2)

33 신장 하나를 기증하면 남은 신장이 보상해 보통 기증 전의 60~70%가 된다. 일반 기준만 보여 주면 정상적인 회복도 "감소"로 보여 불안을 키운다. (20.4)

34 소금: 같은 비밀번호도 사람마다 저장값이 달라져 미리 계산한 표로 풀 수 없다. 반복: 한 번 계산이 느려 무작위 대입이 어렵다. (21.2)

35 401 = 로그인하지 않음(세션 없음, 로그아웃 뒤). 403 = 로그인했지만 허락 안 됨(공여자가 다른 공여자 기록 · 의료진 API 요청). (21.4)

36 병원마다 검사 이름 표기가 달라도 LOINC 코드는 같아서, 어느 병원 자료든 같은 규칙으로 해석할 수 있다. (22.1)

37 저장 전에 같은 날짜 결과를 읽어 값이 같으면 same으로 건너뛰고, 다르면 updated로 덮어쓴다. 같은 날짜는 UNIQUE + ON CONFLICT로 한 행만. (22.3)

38 eCRF는 정해진 변수 이름과 숫자 코드로 입력 · 검증 · 분석한다. Int64는 빈칸(NA)을 허용하는 정수형이라 빈칸이 있어도 오류 없이 "1"로 써진다. (23.2)

39 건강정보(민감정보)의 국외 이전 → 국외이전 동의를 받거나, 이름 등 식별 정보를 빼고 공여자 번호만 보내거나, 국내 · 병원 내부 모델(Ollama)을 쓴다. (25.1)